
PreencheFacil

Release v0.0.3

Avance Telecom

Mar 13, 2026

CONTENTS:

1	text_extractor	3
1.1	app module	3
1.2	colors module	8
1.3	config module	10
1.4	dialogs module	14
1.5	excel_io module	15
1.6	formatters module	17
1.7	icon_manager module	19
1.8	license_dialog module	20
1.9	loading_screen module	21
1.10	models module	22
1.11	online_license module	23
1.12	simple_lock module	26
1.13	user_login module	30
1.14	widgets module	31
	Python Module Index	33
	Index	35

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

TEXT_EXTRACTOR

1.1 app module

`app.abs_path(p)`

Resolve um caminho de arquivo para um caminho absoluto.

Esta função é crucial para lidar com recursos em um aplicativo que pode ser executado tanto em um ambiente de desenvolvimento quanto como um executável “congelado” (por exemplo, com PyInstaller).

- Se o caminho já for absoluto, ele é retornado sem modificação.
- **Se o caminho for relativo, a função determina o diretório base:**
 - Em um executável PyInstaller, usa o diretório temporário `_MEIPASS`.
 - Em um script Python normal, usa o diretório do próprio script.
- O caminho relativo é então combinado com este diretório base.

Parameters

p (`str`) – O caminho do arquivo a ser resolvido.

Returns

O caminho absoluto correspondente, ou uma string vazia se a entrada for nula/vazia.

Return type

`str`

`app.file_exists(p)`

Verifica se um arquivo existe e pode ser lido.

Parameters

p (`str`) – String que representa o caminho para o arquivo a ser verificado.

Returns

True se o arquivo existe e pode ser lido, False caso contrário.

Return type

`bool`

`class app.SettingsDialog(parent)`

Bases: `QDialog`

Classe de diálogo para gerenciar as configurações globais do aplicativo, permitindo ao usuário acessar os ajustes de tema visual e a lista de domínios de e-mail sugeridos.

Parameters

QDialog (`_type_`) – Objeto do tipo `QDialog`.

class app.App(*args: Any, **kwargs: Any)

Bases: QWidget

Interface principal do aplicativo “Preenche Fácil”. Gerencia a configuração de campos, a interface de usuário dinâmica, a integração com planilhas Excel e a persistência de dados.

Parameters

QWidget (_type_) – Objeto do tipo QWidget.

open_license()

Abre o diálogo de informações da licença, exibindo o texto da licença e aplicando o tema visual atual do aplicativo.

Return type

None

open_email_domains()

Abre o diálogo de gerenciamento de domínios de e-mail. Permite adicionar, editar ou remover domínios da lista de sugestões. Ao aceitar, salva as alterações e atualiza todos os widgets de e-mail ativos.

Return type

None

clear_selected_sheet()

Limpa a seleção da planilha atual sem apagar o arquivo do disco.

Remove o vínculo da interface com a planilha, limpa a lista de abas, atualiza a configuração persistida e mantém os campos do controle na UI.

Return type

None

on_sheet_changed(new_sheet)

Atualiza a aba ativa e recarrega os campos correspondentes. Se a aba for nova, tenta sincronizar os cabeçalhos a partir do arquivo Excel.

Parameters

new_sheet (str) – String representando a nova aba

Return type

None

create_new_sheet()

Função responsável por criar uma nova aba na planilha

Return type

None

refresh_sheets()

Atualiza a lista de abas (sheets) disponíveis no arquivo Excel selecionado e mantém o estado da aba atual consistente com o conteúdo do arquivo.

Return type

None

open_settings()

Função responsável por abrir o diálogo de configurações

Return type

None

open_help()

Acessa a página web com informações sobre o software

Return type

None

render_fields()

Renderiza a lista de campos na interface, recriando as linhas de entrada.

Preserva os valores já digitados (incluindo partes do e-mail), limpa o layout atual e monta novamente os widgets conforme o tipo de cada campo (email, booleano ou texto), conectando sinais de editar/excluir e aplicando ícones e estado de bloqueio.

Return type

None

Returns

None

sync_fields_from_existing_excel(path)

Sincroniza os campos do controle com o cabeçalho existente na planilha.

Lê os cabeçalhos da aba, cria cabeçalho se estiver ausente, ou então mapeia cada coluna para um Campo (reaproveitando id/tipo quando possível e inferindo para novos títulos). Ao final, persiste a configuração, aplica regras de tipos de coluna e re-renderiza os campos na UI.

Parameters

path (str) – Caminho do arquivo Excel.

Returns

True se a sincronização foi concluída com sucesso; False em caso de erro ou ausência de cabeçalho válido.

Return type

bool

choose_file()

Abre o diálogo para escolher/criar uma planilha Excel e aplica a seleção.

Se o arquivo já existir, pergunta se o usuário deseja importar os campos a partir do cabeçalho (linha 1). Caso aceite, sincroniza os campos do app com a planilha; caso contrário, mantém os campos atuais e prepara a planilha.

Return type

None

Returns

None

on_field_lock_toggled(field_id, locked)

Atualiza o estado de bloqueio de um campo e persiste a alteração.

Localiza o Campo pelo *field_id*, ajusta o atributo *locked*, salva a configuração e re-renderiza a lista de campos para refletir o novo estado.

Parameters

- **field_id** (str) – ID do campo cujo bloqueio foi alterado.
- **locked** (bool) – Novo estado de bloqueio (True = bloqueado).

Return type

None

Returns

None

add_field()

Adiciona um novo campo de preenchimento.

Solicita ao usuário o título e o tipo do campo, valida duplicidade de título, gera um ID único, salva no controle, re-renderiza a UI e (se houver planilha selecionada) tenta garantir/atualizar as colunas no Excel.

Return type

None

Returns

None

edit_field(*field_id*)

Edita o título e/ou o tipo de um campo existente.

Impede edição de campos bloqueados, abre diálogos para alterar título e tipo, valida duplicidade, persiste a mudança e tenta refletir no Excel (renomeando o cabeçalho e reaplicando regras de tipos). Após re-renderizar, restaura o valor previamente digitado, ajustando-o quando há troca entre tipos com/sem máscara (telefone/data) e e-mail.

Parameters

field_id (str) – ID do campo a ser editado.

Return type

None

Returns

None

delete_field(*field_id*)

Exclui um campo do controle (e a coluna correspondente no Excel, quando possível).

Bloqueia a exclusão de campos protegidos, solicita confirmação do usuário, remove o campo da lista, persiste a alteração e tenta remover a coluna na planilha e reaplicar as regras de tipos. Em seguida, re-renderiza a UI e atualiza o status.

Parameters

field_id (str) – ID do campo a ser excluído.

Return type

None

Returns

None

generate_protocol()

Gera um número de protocolo no formato ddMMyyHHmmss. Se existir algum campo do tipo 'protocolo', preenche automaticamente o primeiro campo encontrado com o valor gerado. Não cria campo novo automaticamente.

Return type

None

delete_all_fields()

Exclui todos os campos não protegidos do controle e, quando possível, remove também as colunas correspondentes na planilha Excel.

A função: - verifica se há campos cadastrados; - impede a exclusão de campos protegidos; - solicita confirmação do usuário; - remove do controle apenas os campos permitidos; - persiste a alteração; - tenta remover

as colunas correspondentes no Excel; - reaplica as regras de tipo nas colunas restantes; - re-renderiza a interface e atualiza o status.

Return type

None

Returns

None

clear_fields()

Limpa os valores preenchidos nos campos da interface.

Zera os QLineEdit e, no caso de e-mail, mantém o domínio e limpa apenas a parte local. Também atualiza a mensagem de status.

Return type

None

Returns

None

save_lead()

Salva os dados preenchidos como uma nova linha na planilha.

Valida se há arquivo selecionado, verifica possível bloqueio do Excel, coleta os valores dos campos (tratando e-mail e booleano), normaliza texto com máscara e evita salvar quando tudo estiver vazio. Em seguida, tenta gravar a linha com novas tentativas em caso de PermissionError e, ao concluir, limpa os campos e informa o usuário.

Return type

None

Returns

None

open_theme_settings()

Abre o diálogo de tema e permite ajustar a cor de fundo (HSL) em tempo real.

Aplica uma prévia do tema conforme os sliders mudam. Se o usuário confirmar, salva as novas configurações (background_hsl e theme) e aplica o tema derivado. Se cancelar, restaura o tema/configuração anterior.

Return type

None

Returns

None

app.main()

Ponto de entrada da aplicação.

Inicializa o QApplication e uma tela de carregamento, solicita o e-mail do usuário e valida a licença online (com tentativas quando aplicável). Em seguida, valida/marca o dispositivo localmente e, se tudo estiver ok, instancia e exibe a janela principal.

Return type

None

Returns

None

1.2 colors module

`colors.hex_to_rgb(hex_color)`

Convert a hexadecimal color string to an RGB tuple.

Accepts strings in the form #RRGGBB or RRGGBB. If the input is not a 6-digit hex color, returns (0, 0, 0).

Parameters

hex_color (str) – Hex color string.

Return type

Tuple[int, int, int]

Returns

A tuple (r, g, b) with values in the range 0..255.

`colors.rgb_to_hex(r, g, b)`

Convert RGB components to a hexadecimal color string.

Components are clamped to the range 0..255.

Parameters

- **r** (int) – Red component (0..255).
- **g** (int) – Green component (0..255).
- **b** (int) – Blue component (0..255).

Return type

str

Returns

Uppercase hex string in the form #RRGGBB.

`colors.rgb_to_hsl(r, g, b)`

Convert RGB (0..255) to HSL-like integers.

Internally uses Python's `colorsys.rgb_to_hls` which returns (H, L, S). This function returns an integer tuple that matches your UI needs:

- H: 0..359 (degrees)
- S: 0..100 (percent)
- L: 0..100 (percent)

Parameters

- **r** (int) – Red component (0..255).
- **g** (int) – Green component (0..255).
- **b** (int) – Blue component (0..255).

Return type

Tuple[int, int, int]

Returns

Tuple (h, s, l) with integer values.

`colors.hsl_to_rgb(h, s, l)`

Convert HSL-like integers to RGB (0..255).

This function expects H in degrees and S/L in percent. Values are clamped where appropriate, and H is normalized with modulo 360.

Note: `colors.hls_to_rgb` expects (H, L, S), hence the parameter order.

Parameters

- **h** (int) – Hue in degrees (any int; normalized to 0..359).
- **s** (int) – Saturation in percent (0..100).
- **l** (int) – Lightness in percent (0..100).

Return type

Tuple[int, int, int]

Returns

Tuple (r, g, b) with integer values 0..255.

`colors.luminance(rgb)`

Compute a simple perceived luminance from an RGB tuple.

Uses weighted coefficients (Rec. 709-like) and normalizes the result to 0..1.

Parameters

rgb (Tuple[int, int, int]) – Tuple (r, g, b) with values 0..255.

Return type

float

Returns

Luminance value in the range 0..1.

`colors.blend(a, b, t)`

Blend two RGB colors.

Linear interpolation between color a and b by factor t.

- **t** = 0 returns a
- **t** = 1 returns b

Parameters

- **a** (Tuple[int, int, int]) – First color as (r, g, b).
- **b** (Tuple[int, int, int]) – Second color as (r, g, b).
- **t** (float) – Blend factor in the range 0..1 (clamped).

Return type

Tuple[int, int, int]

Returns

Blended color as (r, g, b).

`colors.derive_theme_from_background(bg_hex, base_theme)`

Derive a full theme palette from a background color.

Given a background color, this function computes: - surface colors (slightly blended toward white/black) - border color - text color (pulled toward white/black but influenced by the background) - muted text color (between text and border)

The algorithm adapts depending on whether the background is considered dark (luminance < 0.5) or light.

Parameters

- **bg_hex** (str) – Background color as #RRGGBB.
- **base_theme** (dict) – Base theme dictionary to copy and override. Must contain keys like text, primary, danger etc. Missing keys are kept as-is.

Returns

background, surface, surface_alt, border, text, and muted_text.

Return type

dict

1.3 config module

`config.default_fields_config()`

Configurações padrão para campos e abas.

Returns

Configurações iniciais com uma aba “Preenche Fácil” contendo campos básicos.

Return type

dict

`config.rename_sheet_key(cfg, old_name, new_name)`

Renomeia uma aba (sheet) dentro da configuração, movendo os campos associados para a nova chave.

Parameters

- **cfg** (dict) – Configuração completa onde as abas estão definidas. Deve conter a chave “abas” que é um dicionário de abas.
- **old_name** (str) – Nome atual da aba que se deseja renomear. Se não existir, a função apenas garante que a nova aba exista sem campos.
- **new_name** (str) – Novo nome para a aba. Se já existir, os campos só serão movidos se a aba de destino estiver vazia. Se for igual ao old_name, não faz nada.

Return type

None

`config.get_sheet_campos(cfg, sheet_name)`

Obtém os campos configurados para uma aba específica dentro da configuração.

Parameters

- **cfg** (dict) – Configuração completa onde as abas estão definidas. Deve conter a chave “abas” que é um dicionário de abas, cada uma com uma lista de campos.
- **sheet_name** (str) – Nome da aba para a qual se deseja obter os campos. Se a aba não existir, retorna uma lista vazia.

Returns

Lista de campos configurados para a aba especificada. Cada campo é um dicionário com chaves como “id”, “titulo”, “tipo” e “fixo”. Se a aba ou os campos não estiverem definidos, retorna uma lista vazia.

Return type

list

`config.set_sheet_campos(cfg, sheet_name, campos)`

Define os campos para uma aba específica dentro da configuração, normalizando a lista de campos e garantindo a estrutura correta.

Parameters

- **cfg** (dict) – Configuração completa onde as abas estão definidas. Deve conter a chave “abas” que é um dicionário de abas, cada uma com uma lista de campos.
- **sheet_name** (str) – Nome da aba para a qual se deseja definir os campos. Se a aba não existir, será criada automaticamente.
- **campos** (list) – Lista de campos a ser definida para a aba especificada. Cada campo deve ser um dicionário, mas o formato é flexível. A função irá normalizar os campos, garantindo que cada um tenha as chaves mínimas e tipos válidos, conforme definido na função `_normalize_campos`.

Return type

None

`config.load_fields_config(field_types)`

Carrega a configuração de campos e abas a partir do arquivo JSON, aplicando normalização e migração de formato se necessário.

Parameters

field_types (List[str]) – Lista de tipos de campo válidos para normalização. Usada para garantir que os campos carregados tenham tipos reconhecidos, definindo como “texto” se o tipo for inválido ou ausente.

Returns

Configuração de campos carregada e normalizada.

Return type

dict

`config.save_fields_config(cfg)`

Salva a configuração de campos e abas em um arquivo JSON, mantendo a estrutura esperada.

Parameters

cfg (dict) – Configuração de campos a ser salva. Deve conter a estrutura esperada, incluindo “abas” como um dicionário de abas, cada uma com uma lista de campos. A função não realiza validação ou normalização, portanto, é responsabilidade do chamador garantir que a configuração esteja no formato correto antes de chamar esta função.

Return type

None

`config.load_sheet_config()`

Carrega a configuração de planilha a partir do arquivo JSON. Se o arquivo não existir ou ocorrer um erro durante a leitura, retorna um dicionário vazio. A função não realiza validação ou normalização da configuração, portanto, é responsabilidade do chamador garantir que a configuração esteja no formato correto após o carregamento.

Returns

Configuração de planilha carregada a partir do arquivo JSON. Se o arquivo não existir ou ocorrer um erro durante a leitura, retorna um dicionário vazio. A função não realiza validação ou normalização da configuração, portanto, é responsabilidade do chamador garantir que a configuração esteja no formato correto após o carregamento.

Return type

dict

config.save_sheet_config(path)

Salva o caminho da planilha na configuração, escrevendo em um arquivo JSON. A função irá criar ou sobrescrever o arquivo de configuração com um dicionário contendo a chave “last_sheet_path” e o valor do caminho fornecido. A função não realiza validação do caminho fornecido, portanto, é responsabilidade do chamador garantir que o caminho seja válido e acessível antes de chamar esta função.

Parameters

path (str) – Caminho da planilha a ser salvo na configuração. A função irá salvar este caminho no arquivo de configuração JSON, substituindo o valor existente. A função não realiza validação do caminho fornecido, portanto, é responsabilidade do chamador garantir que o caminho seja válido e acessível antes de chamar esta função.

Return type

None

config.get_last_sheet_path()

Obtém o caminho da última planilha salvo na configuração. A função lê o arquivo de configuração JSON para obter o valor associado à chave “last_sheet_path”. Se o arquivo de configuração não existir, se a chave “last_sheet_path” não estiver presente ou se o caminho armazenado não corresponder a um arquivo existente no sistema de arquivos, a função retorna None. Caso contrário, retorna o caminho da última planilha como uma string.

Returns

Caminho da última planilha salvo na configuração. Se o caminho não estiver presente na configuração ou se o arquivo de configuração não existir, retorna None. Se o caminho estiver presente mas o arquivo correspondente não existir no sistema de arquivos, também retorna None. Caso contrário, retorna o caminho da última planilha como uma string.

Return type

Optional[str]

config.default_ui_config()

Configurações padrão para a interface do usuário, incluindo ícones, tema de cores e HTML para o rodapé. Esta configuração é usada como base para a personalização da aparência da aplicação. A função retorna um dicionário contendo as seguintes chaves:

Returns

Configurações padrão para a interface do usuário, incluindo ícones, tema de cores e HTML para o rodapé. Esta configuração é usada como base para a personalização da aparência da aplicação. A função retorna um dicionário contendo as seguintes chaves:

Return type

dict

- “window_icon”: Caminho para o ícone da janela principal.
- “button_icons”: Dicionário de caminhos para ícones de botões específicos, como “choose_file”, “add_field”, “save_lead”, “clear”, “edit_title”, “delete_field” e “settings”.
- “theme”: Dicionário de cores para a interface, incluindo “background”, “surface

“, “surface_alt”, “text”, “muted_text”, “primary”, “danger” e “border”. - “background_hsl”: Dicionário com valores de matiz (h), saturação (s) e luminosidade (l) para a cor de fundo, usado para ajustes dinâmicos de tema. - “footer_left_html”: String contendo HTML para o conteúdo do rodapé à esquerda, permitindo personalização de mensagens ou links.

config.load_ui_config()

Carrega as configurações da interface do usuário a partir de um arquivo JSON. Se o arquivo não existir ou

ocorrer um erro durante a leitura, retorna as configurações padrão definidas pela função `default_ui_config()`. A função garante que as chaves principais estejam presentes na configuração carregada, preenchendo com valores padrão para quaisquer chaves ausentes. A configuração inclui temas de cores, ícones e HTML para o rodapé, permitindo personalização da aparência da aplicação. :returns: Configurações da interface do usuário carregadas a partir do arquivo JSON. Se o arquivo não existir ou ocorrer um erro durante a leitura, retorna as configurações padrão definidas pela função `default_ui_config()`. A função garante que as chaves principais estejam presentes na configuração carregada, preenchendo com valores padrão para quaisquer chaves ausentes. A configuração inclui temas de cores, ícones e HTML para o rodapé, permitindo personalização da aparência da aplicação.

Return type

dict

`config.save_ui_config(cfg)`

Salva as configurações da interface do usuário em um arquivo JSON. A função recebe um dicionário de configurações e escreve seu conteúdo em um arquivo de configuração JSON, criando ou sobrescrevendo o arquivo existente. A configuração deve conter chaves principais como “window_icon”, “button_icons”, “theme”, “background_hsl” e “footer_left_html”. A função não realiza validação ou normalização da configuração, portanto, é responsabilidade do chamador garantir que a configuração esteja no formato correto antes de chamar esta função. O arquivo JSON resultante terá uma estrutura legível, com indentação para facilitar a edição manual se necessário.

Parameters

cfg (dict) – Configurações da interface do usuário a ser salva. Deve conter as chaves principais como “window_icon”, “button_icons”, “theme”, “background_hsl” e “footer_left_html”. A função não realiza validação ou normalização da configuração, portanto, é responsabilidade do chamador garantir que a configuração esteja no formato correto antes de chamar esta função. A função irá criar ou sobrescrever o arquivo de configuração JSON com o conteúdo fornecido.

Return type

None

`config.default_email_domains_config()`

Configurações padrão para domínios de email, contendo uma lista de domínios comuns usados para preenchimento automático em campos de email. Esta configuração é usada como base para a personalização dos domínios de email disponíveis na aplicação. A função retorna um dicionário com a chave “dominios”, que é uma lista de strings representando os domínios de email padrão, como “@gmail.com”, “@hotmail.com”, “@yahoo.com” e “@outlook.com”.

Returns

Configurações padrão para domínios de email, contendo uma lista de domínios comuns usados para preenchimento automático em campos de email. Esta configuração é usada como base para a personalização dos domínios de email disponíveis na aplicação. A função retorna um dicionário com a chave “dominios”, que é uma lista de strings representando os domínios de email padrão, como “@gmail.com”, “@hotmail.com”, “@yahoo.com” e “@outlook.com”.

Return type

dict

`config.load_email_domains_config()`

Carrega as configurações de domínios de email a partir de um arquivo JSON. Se o arquivo não existir ou ocorrer um erro durante a leitura, retorna as configurações padrão definidas pela função `default_email_domains_config()`. A função garante que a chave “dominios” esteja presente na configuração carregada, preenchendo com valores padrão para quaisquer chaves ausentes. A configuração inclui uma lista de domínios de email usados para preenchimento automático em campos de email, permitindo personalização dos domínios disponíveis na aplicação.

Returns

Configurações de domínios de email carregadas do arquivo JSON ou as configurações padrão

caso o arquivo não exista ou ocorra um erro durante a leitura.

Return type

dict

config.save_email_domains_config(cfg)

Salva as configurações de domínios de email em um arquivo JSON. A função recebe um dicionário de configurações contendo a chave “domínios”, que é uma lista de strings representando os domínios de email. O conteúdo do dicionário é escrito em um arquivo de configuração JSON, criando ou sobrescrevendo o arquivo existente. A função não realiza validação ou normalização da configuração, portanto, é responsabilidade do chamador garantir que a configuração esteja no formato correto antes de chamar esta função. O arquivo JSON resultante terá uma estrutura legível, com indentação para facilitar a edição manual se necessário.

Parameters

cfg (dict) – Configurações de domínios de email a ser salva. Deve conter a chave “domínios”, que é uma lista de strings representando os domínios de email. A função não realiza validação ou normalização da configuração, portanto, é responsabilidade do chamador garantir que a configuração esteja no formato correto antes de chamar esta função. A função irá criar ou sobrescrever o arquivo de configuração JSON com o conteúdo fornecido, mantendo uma estrutura legível com indentação.

Return type

None

config.delete_sheet_key(cfg_fields, sheet_name)

Remove do controle JSON a configuração referente a uma aba específica.

Parameters

- **cfg_fields** (dict) – Configuração carregada do controle de campos.
- **sheet_name** (str) – Nome da aba a ser removida.

Returns

True se removeu algo, False caso contrário.

Return type

bool

1.4 dialogs module

class dialogs.ThemeDialog(parent, h, s, l)

Bases: QDialog

restore_defaults()

Restaura os sliders para os valores padrão do tema.

Return type

None

values()

Retorna os valores atuais dos sliders de cor.

Returns

Valores do tom, saturação e brilho.

Return type

Tuple[int, int, int]

class dialogs.**CursorStartLineEdit**(*args, force_cursor_start=False, **kwargs)

Bases: QLineEdit

QLineEdit que força o cursor para o início ao receber foco/clique. Útil para campos com inputMask (telefone, data) para facilitar colar.

set_force_cursor_start(enabled)

Define se o cursor será forçado para o início ao receber foco ou clique.

Parameters

enabled (bool) – Se True, o cursor será forçado para o início ao receber foco ou clique.

Return type

None

focusInEvent(event)

Sobrescreve o evento de foco para mover o cursor para o início se a opção estiver habilitada.

Parameters

event (_type_) – Evento de foco recebido.

mousePressEvent(event)

Sobrescreve o evento de clique do mouse para mover o cursor para o início se a opção estiver habilitada.
:type event: _type_ :param event: Evento de clique do mouse recebido. :type event: _type_

class dialogs.**EmailDomainsDialog**(parent, domains)

Bases: QDialog

domains()

Retorna a lista de domínios de e-mail configurados no diálogo, garantindo que cada domínio comece com '@' e seja único.

Returns

Lista de domínios de e-mail configurados no diálogo.

Return type

List[str]

1.5 excel_io module

excel_io.list_sheets_with_ids(path)

Lê um arquivo Excel e retorna uma lista de tuplas contendo o ID da planilha e o nome de cada aba presente no arquivo. O método utiliza a biblioteca openpyxl para carregar o arquivo e iterar sobre as planilhas, extraindo o ID (que pode ser obtido de diferentes atributos dependendo da versão do openpyxl) e o título de cada aba, retornando-os em uma lista estruturada. :param path: Caminho do arquivo Excel a ser lido. :type path: str :returns: Lista de tuplas, onde cada tupla contém o ID da planilha (int) e o nome da aba (str) presente no arquivo Excel.

Return type

List[Tuple[int, str]]

excel_io.list_sheets(path)

Return type

list[str]

`excel_io.ensure_sheet_exists(path, sheet_name)`

Return type

None

`excel_io.is_excel_lock_present(xlsx_path)`

Return type

bool

`excel_io.get_sheet_headers(xlsx_path, sheet_name)`

Return type

list[str]

`excel_io.read_headers_from_excel(path, preferred_sheet)`

Lê a linha 1 e retorna (sheet_name_usado, headers, has_header). has_header = True se encontrou pelo menos 1 valor não vazio na linha 1. - Se preferred_sheet não existir, usa a aba ativa.

Return type

Tuple[str, List[str], bool]

`excel_io.write_headers_from_campos(path, sheet_name, campos)`

Garante que a linha 1 tenha cabeçalhos (títulos) baseados em campos do app. Se a aba não existir, cria.

Return type

None

`excel_io.ensure_workbook(path, sheet_name, headers)`

Garante que o arquivo Excel exista, que a aba exista e que a linha 1 tenha os cabeçalhos (títulos) especificados. :param path: Caminho do arquivo Excel. :type path: str :param sheet_name: Nome da aba onde os dados serão escritos. :type sheet_name: str :param headers: Lista de cabeçalhos (títulos) a serem garantidos na linha 1 da aba. Se a aba já existir, os cabeçalhos serão mesclados com os existentes, mantendo a ordem dos novos e sem duplicatas. :type headers: List[str]

Return type

None

`excel_io.apply_column_type_rules(path, sheet_name, campos)`

Aplica regras de formatação e validação de dados nas colunas do Excel com base nos tipos definidos nos campos do app.

Return type

None

`excel_io.parse_br_number(value)`

Converte texto numérico para int ou float, aceitando padrão brasileiro.

Exemplos aceitos: - 10 - 10,5 - 1.234,56

Retorna: - int, se for inteiro - float, se tiver parte decimal - None, se não for número válido

`excel_io.parse_br_date_or_text(value)`

Retorna datetime se a data for válida e suportada pelo Excel. Se for anterior a 1900, retorna o texto original. Se não for uma data válida, retorna o texto original.

`excel_io.append_row_typed(file_path, sheet_name, campos, row_by_title)`

Adiciona uma nova linha na planilha, respeitando o tipo de cada campo.

Regras principais: - texto/email/telefone/... -> grava como texto - booleano -> grava como texto - numero:

- grava como int/float

- sem forçar casas decimais
- **data:**
 - se ano $\geq$ 1900, grava como datetime com formato DD/MM/YYYY
 - se ano $<$ 1900, grava como texto
- campos vazios -> célula vazia

Return type

None

excel_io.delete_column_by_header(*path, sheet_name, header_name*)

Exclui uma coluna do arquivo Excel com base no nome do cabeçalho. O método verifica se o arquivo e a aba existem, se a linha 1 contém o cabeçalho especificado e, se todas as condições forem atendidas, exclui a coluna correspondente ao cabeçalho fornecido. :param path: Caminho do arquivo Excel. :type path: str :param sheet_name: Nome da aba onde a coluna será excluída. :type sheet_name: str :param header_name: Nome do cabeçalho da coluna a ser excluída. :type header_name: str

Return type

None

excel_io.rename_column_header(*path, sheet_name, old_header, new_header*)

Renomeia o cabeçalho de uma coluna no arquivo Excel. O método verifica se o arquivo e a aba existem, se a linha 1 contém o cabeçalho antigo especificado e, se todas as condições forem atendidas, atualiza o valor do cabeçalho para o novo nome fornecido. :param path: Caminho do arquivo Excel. :type path: str :param sheet_name: Nome da aba onde a coluna será renomeada. :type sheet_name: str :param old_header: Nome do cabeçalho atual da coluna a ser renomeada. :type old_header: str :param new_header: Novo nome do cabeçalho da coluna. :type new_header: str

Return type

None

1.6 formatters module

formatters.normalize_masked_text(*value*)

Normaliza um texto que pode conter caracteres de máscara (como em campos de formulário) e retorna uma versão limpa e formatada do texto. O método remove caracteres de máscara comuns, como sublinhados, espaços, hífen, pontos, barras e parênteses, e verifica se o texto resultante contém apenas caracteres de separação. Se o texto contiver apenas caracteres de separação sem letras ou dígitos, ele retorna uma string vazia. Além disso, se o texto contiver uma barra sem dígitos, ou se tiver dígitos mas menos de 8 quando houver uma barra, ele também retorna uma string vazia. Caso contrário, retorna o texto limpo e formatado. :param value: O texto a ser normalizado. :type value: str :returns: O texto normalizado, sem caracteres de máscara e formatado de acordo com as regras descritas.

Return type

str

formatters.digits_only(*value*)

Remove todos os caracteres não numéricos de uma string, retornando apenas os dígitos. O método utiliza uma expressão regular para substituir qualquer caractere que não seja um dígito por uma string vazia, resultando em uma string composta apenas por números. Se a entrada for None ou vazia, o método retorna uma string vazia. :param value: A string de entrada a ser processada. :type value: str :returns: A string contendo apenas os dígitos da entrada original.

Return type

str

`formatters.strip_mask_chars(value)`

Remove caracteres de máscara comuns de uma string, como sublinhados, espaços, hífen, pontos, barras e parênteses, e retorna a string limpa. O método utiliza uma expressão regular para substituir esses caracteres por uma string vazia, resultando em uma versão da string sem os caracteres de máscara. Se a entrada for None ou vazia, o método retorna uma string vazia. :param value: A string de entrada a ser processada, que pode conter caracteres de máscara. :type value: str :returns: A string limpa, sem caracteres de máscara.

Return type

str

`formatters.parse_decimal_br(s)`

Converte uma string que representa um número no formato brasileiro (com vírgula como separador decimal) para um valor float. O método remove quaisquer caracteres que não sejam dígitos, vírgulas, pontos ou sinais de menos, e então processa a string para garantir que o formato seja interpretado corretamente. Se a string contiver tanto vírgula quanto ponto, o método assume que o ponto é um separador de milhar e a vírgula é o separador decimal. Se a string contiver apenas vírgula, ela é tratada como separador decimal. O método tenta converter a string resultante para float e retorna o valor convertido, ou None se a conversão falhar.

Parameters

s (str) – A string contendo um valor numérico no formato brasileiro (com vírgula como separador decimal).

Returns

O valor numérico convertido para float, ou None se a conversão falhar.

Return type

Optional[float]

`formatters.parse_date_any(s)`

Converte uma string que representa uma data em um objeto date do Python, tentando vários formatos comuns de data. O método aceita strings em formatos como dd/mm/yyyy, dd-mm-yyyy, yyyy-mm-dd e yyyy/mm/dd. Ele tenta converter a string para um objeto date usando cada um desses formatos, e retorna o primeiro resultado bem-sucedido. Se a conversão falhar para todos os formatos, o método retorna None. Se a entrada for None ou vazia, ele também retorna None.

Parameters

s (str) – A string contendo uma data no formato dd/mm/yyyy, dd-mm-yyyy, yyyy-mm-dd ou yyyy/mm/dd.

Returns

A data convertida, ou None se a conversão falhar.

Return type

Optional[date]

`formatters.normalize_bool_ptbr(s)`

Normaliza uma string que representa um valor booleano em português, retornando “Sim” para valores que indiquem verdadeiro e “Não” para valores que indiquem falso. O método aceita várias formas de entrada, como “sim”, “s”, “yes”, “y”, “true”, “1” para verdadeiro, e “não”, “nao”, “n”, “no”, “false”, “0” para falso. A comparação é feita de forma case-insensitive e a string de entrada é limpa de espaços antes da comparação. Se a string não corresponder a nenhum dos valores reconhecidos para verdadeiro ou falso, o método retorna a string original limpa. :param s: A string de entrada que representa um valor booleano em português. :type s: str :returns: “Sim” se a string indicar verdadeiro, “Não” se indicar falso, ou a string original limpa se não corresponder a nenhum dos casos anteriores. Retorna None se a entrada for None ou vazia.

Return type

Optional[str]

formatters.cast_value_by_type(*tipo, raw*)

Converte uma string bruta para um valor do tipo especificado, com suporte para tipos como “numero”, “moeda”, “data” e “booleano”. O método processa a string de entrada com base no tipo solicitado, utilizando funções de formatação específicas para cada tipo. Para “numero” e “moeda”, ele tenta converter a string para um float usando o formato brasileiro. Para “data”, ele tenta converter a string para um objeto date. Para “booleano”, ele normaliza a string para “Sim” ou “Não”. Se a conversão falhar ou se o tipo não for reconhecido, o método retorna a string original limpa. :param tipo: O tipo para o qual a string deve ser convertida, como “numero”, “moeda”, “data” ou “booleano”. :type tipo: str :param raw: A string bruta a ser convertida para o tipo especificado. O método irá processar essa string de acordo com as regras de formatação para o tipo solicitado e retornar o valor convertido, ou a string original limpa se a conversão falhar ou se o tipo não for reconhecido. :type raw: str :returns: O valor convertido para o tipo especificado, ou a string original limpa se a conversão falhar ou se o tipo não for reconhecido. Para “numero” e “moeda”, retorna um float; para “data”, retorna um objeto date; para “booleano”, retorna “Sim” ou “Não”.

Return type

Any

1.7 icon_manager module

class icon_manager.IconManager(*ui_cfg, abs_path, file_exists*)

Bases: object

Centraliza: - Resolução de caminho (ex.: PyInstaller _MEIPASS via resolver externo) - Cache de QPixmap - Tint dos ícones (SourceIn) - Aplicação em botões por key (lido do ui_cfg[“button_icons”])

icon_for_key(*key, tint_hex*)

Retorna um QIcon para uma chave de ícone específica, aplicando uma tintura se necessário. O método busca o caminho do ícone correspondente à chave fornecida no dicionário de configuração ui_cfg[“button_icons”], verifica se o arquivo existe e, se for encontrado, carrega o QPixmap usando o método _load_pixmap. Se o QPixmap for carregado com sucesso, ele aplica a tintura usando o método _tint_pixmap e retorna um QIcon criado a partir do pixmap tinturado. Se a chave não for encontrada, se o arquivo não existir ou se o QPixmap for inválido, o método retorna None. :param key: A chave do ícone a ser buscada no dicionário de configuração ui_cfg[“button_icons”] para obter o caminho do arquivo de ícone correspondente. :type key: str :param tint_hex: A cor hexadecimal da tintura a ser aplicada ao ícone, caso o arquivo de ícone seja encontrado e carregado com sucesso. :type tint_hex: str :returns: Um QIcon criado a partir do QPixmap tinturado correspondente à chave fornecida, ou None se a chave não for encontrada, se o arquivo de ícone não existir ou se o QPixmap for inválido.

Return type

Optional[QIcon]

apply_button_icon(*btn, key, tint_hex*)

Aplica um ícone a um QPushButton com base em uma chave de ícone específica, utilizando o método icon_for_key para obter o QIcon correspondente. O método busca o QIcon usando a chave fornecida e, se um ícone válido for retornado, ele é aplicado ao botão usando o método setIcon do QPushButton. Se o ícone não for encontrado ou for inválido, o método não faz nenhuma alteração no botão. :param btn: O botão ao qual o ícone será aplicado. :type btn: QPushButton :param key: A chave do ícone a ser buscada no dicionário de configuração ui_cfg[“button_icons”] para obter o caminho do arquivo de ícone correspondente. :type key: str :param tint_hex: A cor hexadecimal da tintura a ser aplicada ao ícone, caso o arquivo de ícone seja encontrado e carregado com sucesso. :type tint_hex: str

Return type

None

clear_cache()

Limpa o cache de QPixmap armazenados no IconManager. O método remove todas as entradas do dicionário `_pixmap_cache`, liberando a memória ocupada pelos QPixmaps armazenados. Isso pode ser útil para liberar recursos quando os ícones não são mais necessários ou quando se deseja forçar o recarregamento dos ícones a partir dos arquivos.

Return type

None

1.8 license_dialog module

license_dialog.abs_path(p)

Resolve um caminho de arquivo relativo para um caminho absoluto, considerando o diretório base do aplicativo ou o diretório de execução PyInstaller. O método verifica se o caminho fornecido é absoluto e, se for, retorna-o diretamente. Caso contrário, ele determina o diretório base usando a variável `sys._MEIPASS` (que é definida pelo PyInstaller durante a execução) ou o diretório do arquivo atual, e então combina esse diretório base com o caminho relativo fornecido para obter o caminho absoluto. Se a entrada for None ou vazia, o método retorna uma string vazia. :param p: O caminho relativo do arquivo. :type p: str :returns: O caminho absoluto do arquivo, considerando o diretório base do aplicativo ou o diretório de execução PyInstaller.

Return type

str

license_dialog.read_license_text(*preferred_names=None, fallback_message='Licença não encontrada.\n\nInclua um arquivo LICENSE.pt-BR.txt (ou LICENSE.txt / LICENSE) junto ao instalador.'*)

Return type

str

Lê o conteúdo de um arquivo de licença a partir de uma lista de nomes de arquivos preferenciais, retornando o conteúdo do primeiro arquivo encontrado. O método itera sobre a lista de nomes de arquivos fornecida (ou uma lista padrão se None for fornecida) e tenta ler o conteúdo de cada arquivo usando o caminho absoluto resolvido. Se um arquivo for encontrado e lido com sucesso, seu conteúdo é retornado. Se nenhum dos arquivos na lista for encontrado ou se ocorrer um erro ao ler os arquivos, o método retorna a mensagem de fallback especificada. Args:

`preferred_names` (Optional[list[str]], optional): Lista de nomes de arquivos de licença a serem verificados em ordem de preferência. O método tentará ler o conteúdo do arquivo de licença usando cada nome na lista até encontrar um arquivo existente e legível. Se nenhum dos arquivos na lista for encontrado ou se ocorrer um erro ao ler os arquivos, o método retornará a mensagem de fallback. Se a lista for None ou vazia, o método usará uma lista padrão de nomes de arquivos de licença para verificar. `fallback_message` (str, optional): Defaults to ("Licença não encontrada.

“Inclua um arquivo LICENSE.pt-BR.txt (ou LICENSE.txt / LICENSE) junto ao instalador.”).

Returns:

str: O conteúdo do arquivo de licença encontrado, ou a mensagem de fallback se nenhum arquivo for encontrado.

class license_dialog.LicenseDialog(*parent=None, theme=None, license_text=""*)

Bases: QDialog

Diálogo para exibir o texto da licença do aplicativo, com suporte para personalização de tema e leitura de arquivos de licença a partir de uma lista de nomes preferenciais. O diálogo é modal e inclui um QTextEdit para exibir o conteúdo da licença, bem como um QDialogButtonBox com um botão de

fechar. O método `apply_theme` permite personalizar as cores do diálogo com base em um dicionário de tema fornecido.

Parameters

QDialog (*_type_*) – Diálogo para exibir o texto da licença do aplicativo, com suporte para personalização de tema e leitura de arquivos de licença a partir de uma lista de nomes preferenciais. O diálogo é modal e inclui um `QTextEdit` para exibir o conteúdo da licença, bem como um `QDialogButtonBox` com um botão de fechar. O método `apply_theme` permite personalizar as cores do diálogo com base em um dicionário de tema fornecido.

`apply_theme(theme)`

Aplica um tema personalizado ao diálogo de licença, definindo as cores de fundo, superfície, texto e borda com base nas especificações do dicionário de tema fornecido. O método extrai as cores do dicionário de tema usando as chaves “background”, “surface”, “text” e “border”, e então constrói uma string de estilo CSS para aplicar essas cores aos elementos do diálogo, como o `QDialog`, `QTextEdit` e `QPushButton`. O estilo é aplicado ao diálogo usando o método `setStyleSheet`. :param theme: Dicionário de tema para personalizar as cores do diálogo. O dicionário deve conter as chaves “background”, “surface”, “text” e “border” com valores de cor hexadecimal para personalizar a aparência do diálogo. O método usará essas cores para construir um estilo CSS que será aplicado ao diálogo, definindo as cores de fundo, superfície, texto e borda dos elementos do diálogo. :type theme: dict

Return type

None

1.9 loading_screen module

```
class loading_screen.LoadingScreen(app, title='Iniciando', subtitle='Aguarde enquanto validamos o
acesso...', width=520, height=220, bg='#0B1220', accent='#3B82F6',
text='#E6EDF7', muted='#A7B3C6')
```

Bases: object

Classe para exibir uma tela de carregamento personalizada durante a inicialização do aplicativo, com suporte para mensagens dinâmicas e personalização de cores. O método construtor recebe parâmetros para configurar o título, subtítulo, dimensões e cores da tela de carregamento, e cria um `QPixmap` personalizado usando o método `_make_pixmap`. A classe inclui métodos para mostrar a tela de carregamento, atualizar a mensagem exibida, ocultar a tela e finalizar o splash de forma confiável em relação à janela principal do aplicativo.

`show(message='Preparando...')`

Exibe a tela de carregamento e atualiza a mensagem exibida. O método mostra o splash usando o método `show` do `QSplashScreen`, e então chama o método `update` para definir a mensagem exibida na tela de carregamento. A mensagem é alinhada à parte inferior esquerda da tela usando as flags `Qt.AlignBottom` | `Qt.AlignLeft`, e a cor do texto é definida como a cor de texto secundário configurada na classe. O método processa os eventos do aplicativo para garantir que a interface seja atualizada corretamente. :param message: A mensagem a ser exibida na tela de carregamento. Defaults to “Preparando...”. :type message: str

Return type

None

`update(message)`

Atualiza a mensagem exibida na tela de carregamento. O método utiliza o método `showMessage` do `QSplashScreen` para definir a nova mensagem, alinhando-a à parte inferior esquerda da tela e usando a cor de texto secundário configurada na classe. Após atualizar a mensagem, o método processa os eventos do aplicativo para garantir que a interface seja atualizada e a nova mensagem seja exibida corretamente. :param message: A nova mensagem a ser exibida na tela de carregamento. :type message: str

Return type

None

hide()

Ocultar a tela de carregamento. O método chama o método hide do QSplashScreen para ocultar a tela de carregamento, e então processa os eventos do aplicativo para garantir que a interface seja atualizada corretamente após ocultar o splash.

Return type

None

close()

Fecha a tela de carregamento de forma confiável. O método chama o método close do QSplashScreen para fechar a tela de carregamento, e processa os eventos do aplicativo antes e depois de fechar o splash para garantir que a interface seja atualizada corretamente e que o fechamento ocorra de forma suave.

Return type

None

finish_and_close(main_window)

Fecha o splash de forma confiável. - finish() tenta sincronizar com a janela principal - close() garante o fechamento mesmo se a janela ainda não foi “exposta”

Return type

None

1.10 models module

```
class models.Campo(id, titulo, tipo='texto', fixo=False, locked=False)
```

Bases: object

Representa um campo de extração, com propriedades como id, título, tipo, e flags para indicar se o campo é fixo ou bloqueado. O campo é identificado por um id único gerado a partir do título, e possui um tipo que pode ser inferido a partir do título ou definido explicitamente. A flag “fixo” indica se o campo é obrigatório e não pode ser removido, enquanto a flag “locked” indica se o campo está bloqueado para edição. A classe Campo é usada para definir os campos que serão extraídos dos documentos, e suas propriedades são utilizadas para configurar a extração e a exportação dos dados.

id: str

titulo: str

tipo: str = 'texto'

fixo: bool = False

locked: bool = False

```
models.sanitize_id(title)
```

Gera um ID sanitizado a partir de um título, convertendo-o para minúsculas, substituindo espaços por underscores, e removendo caracteres especiais. O método processa o título de entrada para criar um ID que seja seguro para uso em código e arquivos, garantindo que ele contenha apenas caracteres alfanuméricos e underscores. O resultado é um ID único e legível que pode ser usado para identificar campos de extração de forma consistente. :param title: O título a partir do qual o ID será gerado. O método irá processar essa string para criar um ID sanitizado, convertendo-a para minúsculas, substituindo espaços por underscores, e removendo caracteres especiais, resultando em um ID seguro para uso em código e arquivos. :type title: str :returns: O ID sanitizado gerado a partir do título, contendo apenas caracteres alfanuméricos e underscores, e convertido para minúsculas. O método garante que o ID seja legível e consistente para identificar campos de extração.

Return type

str

`models.make_unique_id(base_id, used)`

Gera um ID único a partir de um ID base, garantindo que ele não esteja presente em um conjunto de IDs já utilizados. O método verifica se o ID base já está no conjunto de IDs utilizados, e se estiver, ele adiciona um sufixo numérico incremental (começando com “_2”) até encontrar um ID que seja único. O ID gerado é adicionado ao conjunto de IDs utilizados para garantir que futuras chamadas ao método também considerem esse ID como utilizado. O resultado é um ID único que pode ser usado para identificar campos de extração ou outros elementos de forma consistente e sem conflitos. :param base_id: O ID base a ser usado como base para gerar um ID único. :type base_id: str :param used: Um conjunto de IDs já utilizados, para garantir unicidade. :type used: set :returns: Um ID único gerado a partir do base_id e do conjunto de IDs utilizados.

Return type

str

`models.infer_type_from_title(header)`

Infere o tipo de um campo a partir do título do campo, utilizando palavras-chave comuns para identificar tipos como “email”, “telefone”, “data”, “moeda”, “numero” e “booleano”. O método processa o título de entrada, convertendo-o para minúsculas e verificando a presença de palavras-chave específicas que indicam cada tipo. Se uma palavra-chave correspondente for encontrada, o método retorna o tipo inferido. Se nenhuma palavra-chave for encontrada, ele retorna “texto” como tipo padrão. :param header: O título do campo a partir do qual o tipo será inferido. :type header: str :returns: O tipo inferido a partir do título do campo.

Return type

str

1.11 online_license module

`online_license.app_state_dir()`

Retorna o caminho do diretório de estado do aplicativo, onde dados como cache de licença podem ser armazenados. O método tenta criar um subdiretório com o nome do aplicativo dentro de vários caminhos base comuns (APPDATA, LOCALAPPDATA, PROGRAMDATA) e verifica se é possível escrever nesse diretório. Se nenhum dos caminhos base permitir a criação de um diretório gravável, o método cria um diretório com o nome do aplicativo dentro do diretório home do usuário. O resultado é um caminho de diretório garantido onde o aplicativo pode armazenar seus dados de estado. :returns: O caminho do diretório de estado do aplicativo, onde dados como cache de licença podem ser armazenados. O método tenta criar um subdiretório com o nome do aplicativo dentro de vários caminhos base comuns (APPDATA, LOCALAPPDATA, PROGRAMDATA) e verifica se é possível escrever nesse diretório. Se nenhum dos caminhos base permitir a criação de um diretório gravável, o método cria um diretório com o nome do aplicativo dentro do diretório home do usuário. O resultado é um caminho de diretório garantido onde o aplicativo pode armazenar seus dados de estado.

Return type

str

`online_license.cache_path()`

Retorna o caminho completo do arquivo de cache de licença, que é construído combinando o diretório de estado do aplicativo (obtido pela função `app_state_dir`) com o nome do arquivo de cache definido na constante `LICENSE_CACHE_FILE`. O resultado é um caminho absoluto para o arquivo de cache onde os dados de licença podem ser armazenados e recuperados. :returns: O caminho completo do arquivo de cache de licença, que é construído combinando o diretório de estado do aplicativo (obtido pela função `app_state_dir`) com o nome do arquivo de cache definido na constante `LICENSE_CACHE_FILE`. O resultado é um caminho absoluto para o arquivo de cache onde os dados de licença podem ser armazenados e recuperados.

Return type

str

online_license.clear_cache()

Limpa o cache de licença removendo o arquivo de cache se ele existir. O método obtém o caminho do arquivo de cache usando a função `cache_path`, verifica se o arquivo existe e, se for encontrado, tenta removê-lo usando a função `os.remove`. Se ocorrer um erro durante a remoção do arquivo, o método ignora a exceção e continua, garantindo que o cache seja limpo sem causar falhas no aplicativo.

Return type

None

online_license.now_ts()

Retorna o timestamp atual em segundos desde a época (1 de janeiro de 1970). O método utiliza a função `time.time` para obter o tempo atual em segundos como um número de ponto flutuante, e então converte esse valor para um inteiro antes de retorná-lo. O resultado é o número de segundos inteiros que se passaram desde a época, representando o momento atual. :returns: O timestamp atual em segundos desde a época (1 de janeiro de 1970), obtido usando a função `time.time` e convertido para um inteiro.

Return type

int

online_license.load_cache()

Carrega o cache de licença a partir do arquivo de cache, retornando um dicionário com os dados armazenados. O método tenta abrir o arquivo de cache usando o caminho obtido pela função `cache_path`, lê seu conteúdo como JSON e retorna o resultado como um dicionário. Se o arquivo não existir ou se ocorrer um erro durante a leitura ou a conversão do JSON, o método retorna um dicionário vazio, indicando que não há dados de cache disponíveis. :returns: O conteúdo do cache de licença carregado a partir do arquivo de cache, ou um dicionário vazio se o arquivo não existir ou se ocorrer um erro ao ler o arquivo. O método tenta abrir o arquivo de cache usando o caminho obtido pela função `cache_path`, lê seu conteúdo como JSON e retorna o resultado como um dicionário. Se o arquivo não existir ou se ocorrer um erro durante a leitura ou a conversão do JSON, o método retorna um dicionário vazio.

Return type

dict

online_license.save_cache(data)

Salva os dados de licença no cache escrevendo um dicionário como JSON em um arquivo de cache. O método recebe um dicionário de dados, converte-o para uma string JSON formatada usando a função `json.dump`, e escreve essa string no arquivo de cache localizado pelo caminho obtido pela função `cache_path`. Se ocorrer um erro durante a escrita do arquivo, o método irá lançar uma exceção, indicando que o processo de salvamento do cache falhou. :param data: O dicionário de dados a ser salvo no cache de licença. O método irá converter esse dicionário para JSON e escrevê-lo no arquivo de cache usando o caminho obtido pela função `cache_path`. Se ocorrer um erro durante a escrita do arquivo, o método irá lançar uma exceção. :type data: dict

Return type

None

online_license.get_machine_guid()

Obtém o valor do MachineGuid do sistema a partir do registro do Windows. O método abre a chave de registro `HKEY_LOCAL_MACHINESOFTWARE\Microsoft\Cryptography`, consulta o valor da chave "MachineGuid" e retorna esse valor como uma string. O MachineGuid é um identificador único para o computador, e é comumente usado para fins de licenciamento e identificação de dispositivos. Se ocorrer um erro ao acessar o registro ou ao obter o valor, o método irá lançar uma exceção, indicando que a obtenção do MachineGuid falhou. :returns: O valor do MachineGuid do sistema, obtido a partir do registro do Windows. O método abre a chave de registro `HKEY_LOCAL_MACHINESOFTWARE\Microsoft\Cryptography`, consulta o valor da chave "MachineGuid" e retorna esse valor como uma string. Se ocorrer um erro ao acessar o registro ou ao obter o valor, o método irá lançar uma exceção.

Return type

str

online_license.device_id()

Gera um identificador de dispositivo único a partir do valor do MachineGuid do sistema, utilizando o hash SHA-256 para criar um identificador seguro e consistente. O método obtém o MachineGuid usando a função `get_machine_guid`, converte esse valor para bytes usando a codificação UTF-8, e então calcula o hash SHA-256 desses bytes usando a função `hashlib.sha256`. O resultado é um hash hexadecimal em letras maiúsculas que representa o identificador único do dispositivo, que pode ser usado para fins de licenciamento e identificação de hardware. :returns: O identificador do dispositivo, gerado a partir do valor do MachineGuid do sistema. O método obtém o MachineGuid usando a função `get_machine_guid`, converte esse valor para bytes usando a codificação UTF-8, e então calcula o hash SHA-256 desses bytes usando a função `hashlib.sha256`. O resultado é um hash hexadecimal em letras maiúsculas que representa o identificador único do dispositivo, que pode ser usado para fins de licenciamento e identificação de hardware.

Return type

str

online_license.call_api(action, email, dev_id)

Faz uma chamada POST para a API de licenciamento, enviando um payload JSON com a ação, o e-mail do usuário e o identificador do dispositivo. O método constrói um dicionário de payload contendo os parâmetros fornecidos, e então utiliza a função `requests.post` para enviar esse payload como JSON para a URL da API definida na constante `API_URL`. A chamada tem um timeout de 15 segundos para evitar bloqueios prolongados. Após receber a resposta, o método verifica o status HTTP e o tipo de conteúdo da resposta para garantir que seja uma resposta JSON válida. Se a resposta indicar sucesso (status code 200 e Content-Type contendo "application/json"), o método retorna o conteúdo da resposta como um dicionário. Caso contrário, ele lança uma exceção `RuntimeError` com uma mensagem apropriada indicando o erro ocorrido. :param action: A ação a ser realizada na API, como "activate" ou "renew", que indica o tipo de operação de licenciamento :type action: str :param email: O e-mail do usuário para o qual a licença está sendo verificada ou ativada. O método inclui esse e-mail no payload enviado para a API, e é usado para identificar o usuário associado à licença. :type email: str :param dev_id: O identificador do dispositivo, gerado a partir do MachineGuid do sistema, que é enviado no payload para a API de licenciamento. Esse identificador é usado para associar a licença a um dispositivo específico e garantir que as verificações de licença sejam feitas corretamente com base no hardware do usuário. :type dev_id: str :raises RuntimeError: Se a resposta da API indicar um erro, como um status HTTP diferente de 200 ou um Content-Type que não contenha "application/json", o método lança uma exceção `RuntimeError` com uma mensagem que descreve o erro ocorrido, incluindo o status code e o conteúdo da resposta para facilitar a depuração. Se ocorrer um erro durante a chamada HTTP, como um timeout ou uma falha de conexão, a exceção correspondente será propagada para o chamador, indicando que a comunicação com a API falhou. :raises RuntimeError: Se a resposta da API indicar um erro, como um status HTTP diferente de 200 ou um Content-Type que não contenha "application/json", o método lança uma exceção `RuntimeError` com uma mensagem que descreve o erro ocorrido, incluindo o status code e o conteúdo da resposta para facilitar a depuração. Se ocorrer um erro durante a chamada HTTP, como um timeout ou uma falha de conexão, a exceção correspondente será propagada para o chamador, indicando que a comunicação com a API falhou.

Returns

O conteúdo da resposta da API de licenciamento, retornado como um dicionário. O método faz uma chamada POST para a API, enviando um payload JSON com a ação, o e-mail do usuário e o identificador do dispositivo. Após receber a resposta, o método verifica o status HTTP e o tipo de conteúdo para garantir que seja uma resposta JSON válida. Se a resposta indicar sucesso (status code 200 e Content-Type contendo "application/json"), o método retorna o conteúdo da resposta como um dicionário. Caso contrário, ele lança uma exceção `RuntimeError` com uma mensagem apropriada indicando o erro ocorrido.

Return type

dict

online_license.ensure_online_license(email)

Verifica a validade da licença online para um usuário específico, utilizando o e-mail do usuário para consultar a API de licenciamento e retornando uma tupla com o resultado da verificação. O método processa o e-mail

de entrada, obtém o identificador do dispositivo usando a função `device_id`, e então verifica se há um cache de licença válido que corresponda ao e-mail e ao dispositivo. Se um cache válido for encontrado, ele retorna um resultado positivo indicando que a licença é válida (`True`) junto com uma mensagem de “OK (cache)” e um código de status “ok_cache”. Se não houver um cache válido, o método determina a ação apropriada (“activate” ou “renew”) com base no estado do cache, e faz uma chamada para a API usando a função `call_api`. Dependendo da resposta da API, o método salva os dados no cache se a licença for válida, ou retorna mensagens de erro específicas para diferentes condições de falha (como “no_license”, “blocked”, “device_limit”, etc.). O resultado final é uma tupla que indica se a licença é válida, uma mensagem descritiva do resultado, e um código de status que pode ser usado para identificar o resultado específico da verificação.

Parameters

email (`str`) – O e-mail do usuário para o qual a licença está sendo verificada ou ativada. O método inclui esse e-mail no payload enviado para a API, e é usado para identificar o usuário associado à licença.

Returns

Uma tupla contendo três elementos: um booleano indicando se a licença é válida (`True`) ou não (`False`), uma string com uma mensagem descritiva do resultado da verificação da licença, e uma string com um código de status que pode ser usado para identificar o resultado específico da verificação (como “ok”, “no_license”, “blocked”, etc.). O método realiza a verificação da licença online, utilizando o e-mail fornecido para consultar a API de licenciamento, e retorna os resultados dessa verificação na forma de uma tupla estruturada.

Return type

`Tuple[bool, str, str]`

1.12 simple_lock module

`simple_lock.exe_dir_base()`

Retorna o diretório base do executável, considerando o ambiente de execução (PyInstaller ou desenvolvimento). O método verifica se o atributo `sys._MEIPASS` está presente, o que indica que o aplicativo está sendo executado a partir de um pacote criado pelo PyInstaller, e retorna esse diretório como base. Caso contrário, ele retorna o diretório do arquivo atual como base. O resultado é o caminho do diretório onde o aplicativo está sendo executado, que pode ser usado para localizar recursos e arquivos relacionados ao aplicativo. :returns: O diretório base do executável, considerando o ambiente de execução (PyInstaller ou desenvolvimento). O método verifica se o atributo `sys._MEIPASS` está presente, o que indica que o aplicativo está sendo executado a partir de um pacote criado pelo PyInstaller, e retorna esse diretório como base. Caso contrário, ele retorna o diretório do arquivo atual como base. O resultado é o caminho do diretório onde o aplicativo está sendo executado, que pode ser usado para localizar recursos e arquivos relacionados ao aplicativo.

Return type

`str`

`simple_lock.internal_dir()`

Retorna o caminho do diretório interno do aplicativo, que é uma subpasta chamada “_internal” localizada dentro do diretório base do executável. O método utiliza a função `exe_dir_base` para obter o diretório base do executável e, em seguida, combina esse diretório com o nome da subpasta “_internal” para formar o caminho completo do diretório interno. Esse diretório é usado para armazenar arquivos relacionados ao aplicativo que não devem ser acessados ou modificados diretamente pelo usuário, como o marcador de licença e outros arquivos de configuração ou estado. :returns: O caminho do diretório interno do aplicativo, que é uma subpasta chamada “_internal” localizada dentro do diretório base do executável. O método utiliza a função `exe_dir_base` para obter o diretório base do executável e, em seguida, combina esse diretório com o nome da subpasta “_internal” para formar o caminho completo do diretório interno. Esse diretório é usado para armazenar arquivos relacionados ao aplicativo que não devem ser acessados ou modificados diretamente pelo usuário, como o marcador de licença e outros arquivos de configuração ou estado.

Return type

str

simple_lock.internal_marker_path()

Retorna o caminho do arquivo de marcador interno, que é um arquivo chamado “_internal.dat” localizado dentro do diretório interno do aplicativo. O método utiliza a função `internal_dir` para obter o caminho do diretório interno e, em seguida, combina esse diretório com o nome do arquivo de marcador para formar o caminho completo do arquivo de marcador. Esse arquivo é criado pelo instalador e serve como um indicador de que o aplicativo foi instalado corretamente, além de ser usado para armazenar informações relacionadas à licença e ao estado do aplicativo. :returns: O caminho do arquivo de marcador interno, que é um arquivo chamado “_internal.dat” localizado dentro do diretório interno do aplicativo. O método utiliza a função `internal_dir` para obter o caminho do diretório interno e, em seguida, combina esse diretório com o nome do arquivo de marcador para formar o caminho completo do arquivo de marcador. Esse arquivo é criado pelo instalador e serve como um indicador de que o aplicativo foi instalado corretamente, além de ser usado para armazenar informações relacionadas à licença e ao estado do aplicativo.

Return type

str

simple_lock.programdata_dir()

Retorna o caminho do diretório ProgramData específico para o aplicativo, que é uma subpasta com o nome do aplicativo localizada dentro do diretório ProgramData do sistema. O método obtém o caminho do diretório ProgramData usando a variável de ambiente “PROGRAMDATA” (ou um valor padrão se a variável não estiver definida), e então combina esse caminho com o nome do aplicativo para formar o caminho completo do diretório específico do aplicativo dentro do ProgramData. O método também garante que esse diretório exista, criando-o se necessário, para que possa ser usado para armazenar arquivos relacionados ao estado e à configuração do aplicativo. :returns: O caminho do diretório ProgramData específico para o aplicativo, que é uma subpasta com o nome do aplicativo localizada dentro do diretório ProgramData do sistema. O método obtém o caminho do diretório ProgramData usando a variável de ambiente “PROGRAMDATA” (ou um valor padrão se a variável não estiver definida), e então combina esse caminho com o nome do aplicativo para formar o caminho completo do diretório específico do aplicativo dentro do ProgramData. O método também garante que esse diretório exista, criando-o se necessário, para que possa ser usado para armazenar arquivos relacionados ao estado e à configuração do aplicativo.

Return type

str

simple_lock.license_path_programdata()

Retorna o caminho do arquivo de licença localizado no diretório ProgramData específico para o aplicativo. O método utiliza a função `programdata_dir` para obter o caminho do diretório específico do aplicativo dentro do ProgramData, e então combina esse diretório com o nome do arquivo de licença (definido pela constante `PROGRAMDATA_FILE`) para formar o caminho completo do arquivo de licença. Esse arquivo é usado para armazenar informações relacionadas à licença do aplicativo, e o método garante que o caminho retornado seja válido para uso na leitura e escrita de dados relacionados à licença. :returns: O caminho completo do arquivo de licença localizado no diretório ProgramData específico para o aplicativo.

Return type

str

simple_lock.machine_id()

Calcula o hash SHA-256 do MachineGuid do registro do Windows e retorna como uma string em letras maiúsculas. O método chama a função `_machine_guid` para obter o MachineGuid da máquina e, em seguida, calcula o hash SHA-256 desse valor usando a biblioteca `hashlib`. O resultado é convertido para uma string hexadecimal e transformado em letras maiúsculas antes de ser retornado. Esse hash é usado como um identificador único para a máquina, permitindo que o aplicativo associe informações ou licenças a um dispositivo específico de forma segura. :returns: O hash SHA-256 do MachineGuid do registro do Windows, retornado como uma string em letras maiúsculas. O método chama a função `_machine_guid` para obter o MachineGuid da máquina e, em seguida,

calcula o hash SHA-256 desse valor usando a biblioteca hashlib. O resultado é convertido para uma string hexadecimal e transformado em letras maiúsculas antes de ser retornado. Esse hash é usado como um identificador único para a máquina, permitindo que o aplicativo associe informações ou licenças a um dispositivo específico de forma segura.

Return type

str

`simple_lock.machine_id_hash(mid)`

Calcula o hash SHA-256 de um identificador de máquina (Machine ID) fornecido como argumento e retorna o resultado como uma string em letras maiúsculas. O método recebe um valor de identificador de máquina (*mid*) como entrada, codifica esse valor em UTF-8 e calcula o hash SHA-256 usando a biblioteca hashlib. O resultado é convertido para uma string hexadecimal e transformado em letras maiúsculas antes de ser retornado. Esse hash é usado para criar um identificador único e seguro para a máquina, permitindo que o aplicativo associe informações ou licenças a um dispositivo específico de forma segura. :param *mid*: O valor do identificador da máquina (Machine ID) a ser convertido em hash SHA-256. :type *mid*: str :returns: O hash SHA-256 do valor passado como argumento *mid*, retornado como uma string em letras maiúsculas.

Return type

str

class `simple_lock.DATA_BLOB`

Bases: Structure

Representa a estrutura DATA_BLOB usada pela API de criptografia do Windows (DPAPI). Essa estrutura é usada para armazenar dados binários que serão protegidos ou desprotegidos usando as funções CryptProtectData e CryptUnprotectData. A estrutura contém dois campos: *cbData*, que é um valor DWORD que indica o tamanho dos dados em bytes, e *pbData*, que é um ponteiro para os dados binários a serem protegidos ou desprotegidos. Essa estrutura é essencial para a interação com a API de criptografia do Windows, permitindo que os dados sejam manipulados de forma segura e eficiente. :param *ctypes*: A classe DATA_BLOB herda de ctypes.Structure, o que permite que ela seja usada para definir uma estrutura de dados compatível com a API de criptografia do Windows. A estrutura é definida usando a sintaxe de classes do Python, e os campos da estrutura são especificados na lista *_fields_*, onde cada campo é representado por uma tupla contendo o nome do campo e seu tipo de dados correspondente (por exemplo, *wt.DWORD* para *cbData* e *ctypes.POINTER(ctypes.c_byte)* para *pbData*). Essa definição permite que a estrutura seja usada para armazenar e manipular dados binários de forma segura ao interagir com as funções de criptografia do Windows. :type *ctypes*: _type_

cbData

Structure/Union member

pbData

Structure/Union member

`simple_lock.dpapi_protect_localmachine(plaintext)`

Protege um objeto de bytes usando a API de criptografia do Windows (DPAPI) com a opção CRYPTPROTECT_LOCAL_MACHINE, que vincula os dados protegidos à máquina em que foram protegidos. O método recebe um objeto de bytes como entrada, converte esses bytes em uma estrutura DATA_BLOB usando a função *_bytes_to_blob*, e então chama a função CryptProtectData para proteger os dados. O resultado é um novo objeto de bytes que representa os dados protegidos, que pode ser armazenado ou transmitido com segurança. Se ocorrer um erro durante o processo de proteção, o método levanta uma exceção OSError com o código de erro correspondente. :param *plaintext*: :type *plaintext*: bytes :raises OSError: Se ocorrer um erro durante o processo de proteção dos dados usando a API de criptografia do Windows, o método levanta uma exceção OSError com o código de erro correspondente, que pode ser obtido usando *ctypes.get_last_error()* para fornecer informações sobre a natureza do erro ocorrido.

Returns

Um novo objeto de bytes que representa os dados protegidos usando a API de criptografia do

Windows (DPAPI) com a opção CRYPTPROTECT_LOCAL_MACHINE. O método converte os bytes de entrada em uma estrutura DATA_BLOB, chama a função CryptProtectData para proteger os dados, e retorna o resultado como um objeto de bytes. Se ocorrer um erro durante o processo de proteção, o método levanta uma exceção OSError com o código de erro correspondente.

Return type

bytes

`simple_lock.dpapi_unprotect(ciphertext)`

Desprotege um objeto de bytes protegido usando a API de criptografia do Windows (DPAPI). O método recebe um objeto de bytes protegido como entrada, converte esses bytes em uma estrutura DATA_BLOB usando a função `_bytes_to_blob`, e então chama a função `CryptUnprotectData` para desproteger os dados. O resultado é um novo objeto de bytes que representa os dados desprotegidos, que pode ser usado ou processado conforme necessário. Se ocorrer um erro durante o processo de desproteção, o método levanta uma exceção OSError com o código de erro correspondente. :param ciphertext: Um objeto de bytes protegido usando a API de criptografia do Windows (DPAPI). :type ciphertext: bytes :raises OSError: Se ocorrer um erro durante o processo de desproteção dos dados usando a API de criptografia do Windows, o método levanta uma exceção OSError com o código de erro correspondente, que pode ser obtido usando `ctypes.get_last_error()` para fornecer informações sobre a natureza do erro ocorrido.

Returns

Um novo objeto de bytes que representa os dados desprotegidos usando a API de criptografia do Windows (DPAPI). O método recebe um objeto de bytes protegido como entrada, converte esses bytes em uma estrutura DATA_BLOB usando a função `_bytes_to_blob`, chama a função `CryptUnprotectData` para desproteger os dados, e retorna o resultado como um objeto de bytes. Se ocorrer um erro durante o processo de desproteção, o método levanta uma exceção OSError com o código de erro correspondente.

Return type

bytes

`simple_lock.ensure_or_mark(allow_create=False)`

Política final (conforme combinado):

- O instalador cria: <pasta app>_internal_internal.dat (marker somente leitura)
- O app NUNCA escreve em `_internal`
- O marcador por máquina fica em ProgramData e é DPAPI (LocalMachine):

C:\ProgramData\LeadsApp\cache_string.dat

Regras

1. Se marker do instalador não existir => bloqueia (evita rodar app copiado solto).
2. Se o arquivo de ProgramData NÃO existir: - `allow_create=False` => bloqueia pedindo ativação online primeiro. - `allow_create=True` => cria o marcador por máquina (use somente após validação online OK).
3. Se o arquivo existir => valida DPAPI + HMAC + machine_id_hash.

Parameters

allow_create (bool) – Indica se o método deve criar um novo marcador por máquina em ProgramData caso o arquivo de marcador não exista. Se `False`, bloqueia e solicita ativação online. Se `True`, cria o marcador por máquina usando DPAPI (use somente após validação online bem-sucedida).

Return type

Tuple[bool, str]

1.13 user_login module

`user_login.app_state_dir()`

Diretório de estado com fallback real: 1) `%APPDATA%` (Roaming) 2) `%LOCALAPPDATA%` (Local) 3) `%PROGRAMDATA%` LeadsAppuser_rootLeadsApp

Returns

O caminho do diretório de estado do aplicativo, determinado por meio de uma série de tentativas. O método verifica primeiro o diretório de roaming (`%APPDATA%`), seguido pelo diretório local (`%LOCALAPPDATA%`), e finalmente tenta criar um diretório específico para o aplicativo dentro do diretório de dados do programa (`%PROGRAMDATA%`). Se todas as tentativas falharem, o método retorna um caminho de fallback dentro do diretório do usuário.

Return type

str

`user_login.login_path()`

Retorna o caminho completo do arquivo de login do aplicativo. O método constrói o caminho do arquivo de login combinando o diretório de estado do aplicativo, obtido por meio da função `app_state_dir()`, com o nome do arquivo de login definido pela constante `LOGIN_FILE`. O resultado é o caminho completo onde as informações de login do usuário serão armazenadas ou recuperadas. :returns: O caminho completo do arquivo de login do aplicativo.

Return type

str

`user_login.load_login()`

Carrega o e-mail e o timestamp do login salvo.

Returns

Uma tupla contendo o e-mail salvo (ou None se não houver) e o timestamp do login (ou None se não houver).

Return type

Tuple[Optional[str], Optional[int]]

`user_login.save_login(email)`

Salva o e-mail do login com um timestamp atual.

Parameters

email (str) – O e-mail a ser salvo. O método irá limpar espaços em branco e converter o e-mail para letras minúsculas antes de salvar. O e-mail é armazenado junto com um timestamp que indica quando o login foi salvo, permitindo que o aplicativo determine se o login é válido ou expirado com base no tempo decorrido desde o último login.

Return type

None

`user_login.clear_login()`

Limpa o login salvo removendo o arquivo de login. O método tenta excluir o arquivo de login do sistema, que contém as informações de e-mail e timestamp do login. Se o arquivo existir, ele é removido, efetivamente limpando o login salvo. Se ocorrer qualquer erro durante a tentativa de remoção do arquivo (por exemplo, se o arquivo não existir ou se houver problemas de permissão), o método captura a exceção e continua sem bloquear o fluxo do programa, garantindo que a limpeza do login seja realizada de forma segura sem causar interrupções.

Return type

None

user_login.is_login_expired(*ts*)

Verifica se o login é expirado com base no timestamp fornecido. O método compara o timestamp do login com o timestamp atual para determinar se o tempo decorrido desde o último login excede o limite definido por LOGIN_MAX_AGE_SECONDS. Se o timestamp fornecido for None, o método considera o login como expirado. Caso contrário, ele calcula a diferença entre o timestamp atual e o timestamp do login e verifica se essa diferença é maior que o limite de idade máxima do login. :param ts: O timestamp do login a ser verificado. Este valor representa o momento em que o login foi salvo e é usado para determinar se o login é válido ou expirado com base no tempo decorrido desde então. :type ts: Optional[int] :returns: Retorna True se o login for considerado expirado (ou seja, se o timestamp for None ou se o tempo decorrido desde o login exceder o limite definido por LOGIN_MAX_AGE_SECONDS), e False caso contrário (ou seja, se o login ainda for válido).

Return type

bool

user_login.prompt_email(*force=False*)

Exibe um diálogo para o usuário inserir seu e-mail de login. O método verifica se há um e-mail salvo anteriormente e se ele ainda é válido (não expirado). Se houver um e-mail válido salvo, ele é retornado imediatamente, a menos que o parâmetro force seja definido como True, o que força o usuário a inserir um novo e-mail. Se não houver um e-mail válido salvo ou se force for True, o método exibe um diálogo de entrada para o usuário digitar seu e-mail. O método valida o formato do e-mail inserido usando uma expressão regular e continua solicitando até que um e-mail válido seja fornecido ou até que o usuário cancele a operação. Se um e-mail válido for inserido, ele é salvo para uso futuro e retornado pelo método. Se o usuário cancelar a operação, o método retorna None. :param force: Se True, força o usuário a inserir um novo e-mail mesmo que haja um e-mail salvo válido. Defaults to False. :type force: bool :returns: O e-mail inserido pelo usuário se for válido, ou o e-mail salvo anteriormente se ainda for válido, ou None se o usuário cancelar a operação.

Return type

Optional[str]

1.14 widgets module

class widgets.EmailInputWidget(*domains, parent=None*)

Bases: QWidget

Encapsula: - QLineEdit (parte local) + QComboBox (domínio) - Monta e lê email final - Atualiza lista de domínios preservando seleção

- Se o local contém '@', considera colado completo e retorna ok=True
- Se local não vazio e domínio vazio => ok=False
- Se local vazio => retorna "" e ok=True (campo em branco)

set_domains(*domains*)**Return type**

None

local()**Return type**

str

domain()**Return type**

str

set_parts(*local, domain*)

Return type

None

set_email(*email*)

Return type

None

get_email()

Return type

Tuple[str, bool]

class widgets.**FieldRowWidget**(*field_id, label_text, input_widget, parent=None*)

Bases: QWidget

editRequested

alias of str

deleteRequested

alias of str

lockToggled

alias of str

set_locked(*locked*)

Return type

None

class widgets.**BoolInputWidget**(*parent=None*)

Bases: QWidget

Widget de entrada booleana baseado em QComboBox.

value()

Return type

str

set_value(*v*)

Return type

None

PYTHON MODULE INDEX

a

`app`, 3

c

`colors`, 8

`config`, 10

d

`dialogs`, 14

e

`excel_io`, 15

f

`formatters`, 17

i

`icon_manager`, 19

l

`license_dialog`, 20

`loading_screen`, 21

m

`models`, 22

o

`online_license`, 23

s

`simple_lock`, 26

u

`user_login`, 30

w

`widgets`, 31

A

abs_path() (in module *app*), 3
 abs_path() (in module *license_dialog*), 20
 add_field() (*app.App* method), 6
 app
 module, 3
 App (class in *app*), 3
 app_state_dir() (in module *online_license*), 23
 app_state_dir() (in module *user_login*), 30
 append_row_typed() (in module *excel_io*), 16
 apply_button_icon() (*icon_manager.IconManager* method), 19
 apply_column_type_rules() (in module *excel_io*), 16
 apply_theme() (*license_dialog.LicenseDialog* method), 21

B

blend() (in module *colors*), 9
 BoolInputWidget (class in *widgets*), 32

C

cache_path() (in module *online_license*), 23
 call_api() (in module *online_license*), 25
 Campo (class in *models*), 22
 cast_value_by_type() (in module *formatters*), 19
 cbData (*simple_lock.DATA_BLOB* attribute), 28
 choose_file() (*app.App* method), 5
 clear_cache() (*icon_manager.IconManager* method), 19
 clear_cache() (in module *online_license*), 23
 clear_fields() (*app.App* method), 7
 clear_login() (in module *user_login*), 30
 clear_selected_sheet() (*app.App* method), 4
 close() (*loading_screen.LoadingScreen* method), 22
 colors
 module, 8
 config
 module, 10
 create_new_sheet() (*app.App* method), 4
 CursorStartLineEdit (class in *dialogs*), 14

D

DATA_BLOB (class in *simple_lock*), 28
 default_email_domains_config() (in module *config*), 13
 default_fields_config() (in module *config*), 10
 default_ui_config() (in module *config*), 12
 delete_all_fields() (*app.App* method), 6
 delete_column_by_header() (in module *excel_io*), 17
 delete_field() (*app.App* method), 6
 delete_sheet_key() (in module *config*), 14
 deleteRequested (*widgets.FieldRowWidget* attribute), 32
 derive_theme_from_background() (in module *colors*), 9
 device_id() (in module *online_license*), 25
 dialogs
 module, 14
 digits_only() (in module *formatters*), 17
 domain() (*widgets.EmailInputWidget* method), 31
 domains() (*dialogs.EmailDomainsDialog* method), 15
 dpapi_protect_localmachine() (in module *simple_lock*), 28
 dpapi_unprotect() (in module *simple_lock*), 29

E

edit_field() (*app.App* method), 6
 editRequested (*widgets.FieldRowWidget* attribute), 32
 EmailDomainsDialog (class in *dialogs*), 15
 EmailInputWidget (class in *widgets*), 31
 ensure_online_license() (in module *online_license*), 25
 ensure_or_mark() (in module *simple_lock*), 29
 ensure_sheet_exists() (in module *excel_io*), 15
 ensure_workbook() (in module *excel_io*), 16
 excel_io
 module, 15
 exe_dir_base() (in module *simple_lock*), 26

F

FieldRowWidget (class in *widgets*), 32
 file_exists() (in module *app*), 3

finish_and_close() (*loading_screen.LoadingScreen* method), 22
 fixo (*models.Campo* attribute), 22
 focusInEvent() (*dialogs.CursorStartLineEdit* method), 15
 formatters
 module, 17

G

generate_protocol() (*app.App* method), 6
 get_email() (*widgets.EmailInputWidget* method), 32
 get_last_sheet_path() (*in module config*), 12
 get_machine_guid() (*in module online_license*), 24
 get_sheet_campos() (*in module config*), 10
 get_sheet_headers() (*in module excel_io*), 16

H

hex_to_rgb() (*in module colors*), 8
 hide() (*loading_screen.LoadingScreen* method), 22
 hsl_to_rgb() (*in module colors*), 8

I

icon_for_key() (*icon_manager.IconManager* method), 19
 icon_manager
 module, 19
 IconManager (class in *icon_manager*), 19
 id (*models.Campo* attribute), 22
 infer_type_from_title() (*in module models*), 23
 internal_dir() (*in module simple_lock*), 26
 internal_marker_path() (*in module simple_lock*), 27
 is_excel_lock_present() (*in module excel_io*), 16
 is_login_expired() (*in module user_login*), 30

L

license_dialog
 module, 20
 license_path_programdata() (*in module simple_lock*), 27
 LicenseDialog (class in *license_dialog*), 20
 list_sheets() (*in module excel_io*), 15
 list_sheets_with_ids() (*in module excel_io*), 15
 load_cache() (*in module online_license*), 24
 load_email_domains_config() (*in module config*), 13
 load_fields_config() (*in module config*), 11
 load_login() (*in module user_login*), 30
 load_sheet_config() (*in module config*), 11
 load_ui_config() (*in module config*), 12
 loading_screen
 module, 21
 LoadingScreen (class in *loading_screen*), 21
 local() (*widgets.EmailInputWidget* method), 31
 locked (*models.Campo* attribute), 22

lockToggled (*widgets.FieldRowWidget* attribute), 32
 login_path() (*in module user_login*), 30
 luminance() (*in module colors*), 9

M

machine_id() (*in module simple_lock*), 27
 machine_id_hash() (*in module simple_lock*), 28
 main() (*in module app*), 7
 make_unique_id() (*in module models*), 23
 models
 module, 22

module

app, 3
 colors, 8
 config, 10
 dialogs, 14
 excel_io, 15
 formatters, 17
 icon_manager, 19
 license_dialog, 20
 loading_screen, 21
 models, 22
 online_license, 23
 simple_lock, 26
 user_login, 30
 widgets, 31

mousePressEvent() (*dialogs.CursorStartLineEdit* method), 15

N

normalize_bool_ptbr() (*in module formatters*), 18
 normalize_masked_text() (*in module formatters*), 17
 now_ts() (*in module online_license*), 24

O

on_field_lock_toggled() (*app.App* method), 5
 on_sheet_changed() (*app.App* method), 4
 online_license
 module, 23
 open_email_domains() (*app.App* method), 4
 open_help() (*app.App* method), 4
 open_license() (*app.App* method), 4
 open_settings() (*app.App* method), 4
 open_theme_settings() (*app.App* method), 7

P

parse_br_date_or_text() (*in module excel_io*), 16
 parse_br_number() (*in module excel_io*), 16
 parse_date_any() (*in module formatters*), 18
 parse_decimal_br() (*in module formatters*), 18
 pbData (*simple_lock.DATA_BLOB* attribute), 28
 programdata_dir() (*in module simple_lock*), 27
 prompt_email() (*in module user_login*), 31

R

[read_headers_from_excel\(\)](#) (in module *excel_io*), 16
[read_license_text\(\)](#) (in module *license_dialog*), 20
[refresh_sheets\(\)](#) (*app.App* method), 4
[rename_column_header\(\)](#) (in module *excel_io*), 17
[rename_sheet_key\(\)](#) (in module *config*), 10
[render_fields\(\)](#) (*app.App* method), 5
[restore_defaults\(\)](#) (*dialogs.ThemeDialog* method), 14
[rgb_to_hex\(\)](#) (in module *colors*), 8
[rgb_to_hsl\(\)](#) (in module *colors*), 8

S

[sanitize_id\(\)](#) (in module *models*), 22
[save_cache\(\)](#) (in module *online_license*), 24
[save_email_domains_config\(\)](#) (in module *config*), 14
[save_fields_config\(\)](#) (in module *config*), 11
[save_lead\(\)](#) (*app.App* method), 7
[save_login\(\)](#) (in module *user_login*), 30
[save_sheet_config\(\)](#) (in module *config*), 11
[save_ui_config\(\)](#) (in module *config*), 13
[set_domains\(\)](#) (*widgets.EmailInputWidget* method), 31
[set_email\(\)](#) (*widgets.EmailInputWidget* method), 32
[set_force_cursor_start\(\)](#) (*dialogs.CursorStartLineEdit* method), 15
[set_locked\(\)](#) (*widgets.FieldRowWidget* method), 32
[set_parts\(\)](#) (*widgets.EmailInputWidget* method), 31
[set_sheet_campos\(\)](#) (in module *config*), 10
[set_value\(\)](#) (*widgets.BoolInputWidget* method), 32
[SettingsDialog](#) (class in *app*), 3
[show\(\)](#) (*loading_screen.LoadingScreen* method), 21
[simple_lock](#)
 module, 26
[strip_mask_chars\(\)](#) (in module *formatters*), 18
[sync_fields_from_existing_excel\(\)](#) (*app.App* method), 5

T

[ThemeDialog](#) (class in *dialogs*), 14
[tipo](#) (*models.Campo* attribute), 22
[titulo](#) (*models.Campo* attribute), 22

U

[update\(\)](#) (*loading_screen.LoadingScreen* method), 21
[user_login](#)
 module, 30

V

[value\(\)](#) (*widgets.BoolInputWidget* method), 32
[values\(\)](#) (*dialogs.ThemeDialog* method), 14

W

[widgets](#)